COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | 9/8/26 | **DATE SUBMITTED:** | |
| **NAME:** | Nichollo Dave A. Galvez | **INSTRUCTOR:** | Engr. Jimlord Quejado |

**ACTIVITY NO. ___ [ACTIVITY NAME]**

**1. PROCEDURE OUTPUT**

| Table 7-1 | Array of Values for Sort Algorithm Testing |
|---|---|
| Code and Output: | |
| Observation: | The code creates a 100-element array and fills it with random numbers between 0 and 99 using rand(). A helper function then prints the full, space-separated list, giving you a randomized set of numbers to confirm everything worked properly. |

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

//bubble sort

// generate 100 random values
// create a function that will display the random generated values

const int maxSize = 100;

void displayArr(int arr[]);

int main() {
    // create an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    // use time(0) as seed for random values
    std::srand(std::time(0));

    // populate the array with random values
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = std::rand() % 100;
    }

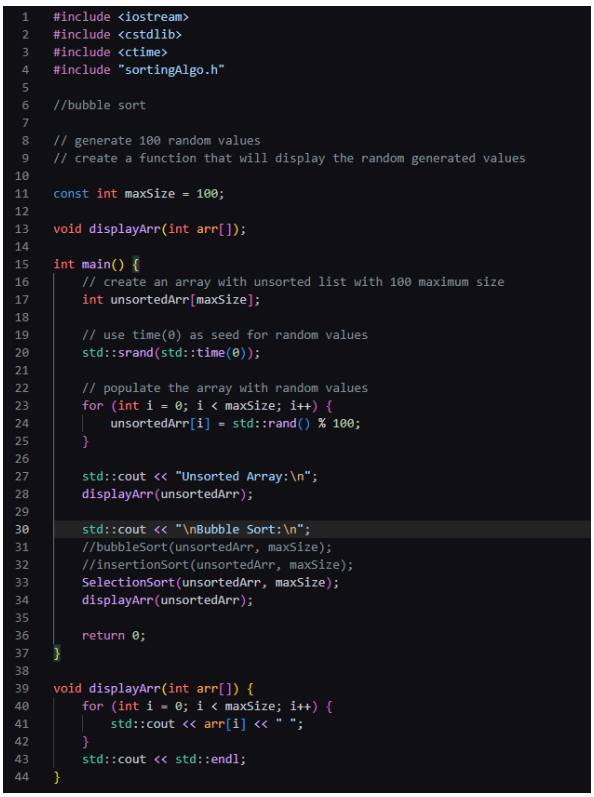
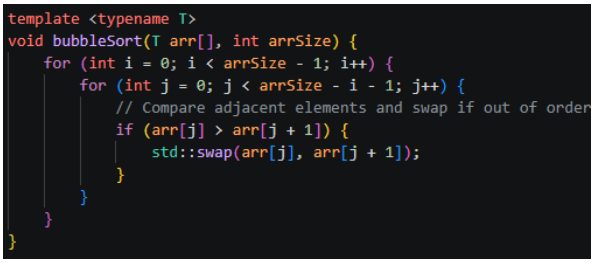
    std::cout << "Unsorted Array:\n";
    displayArr(unsortedArr);

    std::cout << "\nInsertion Sort:\n";
    //bubbleSort(unsortedArr, maxSize);
    //insertionSort(unsortedArr, maxSize);
    SelectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

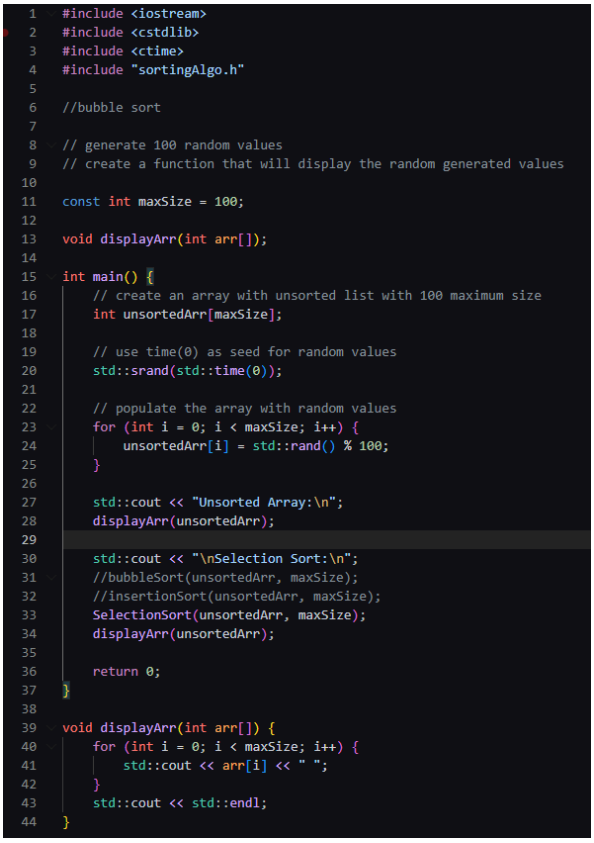
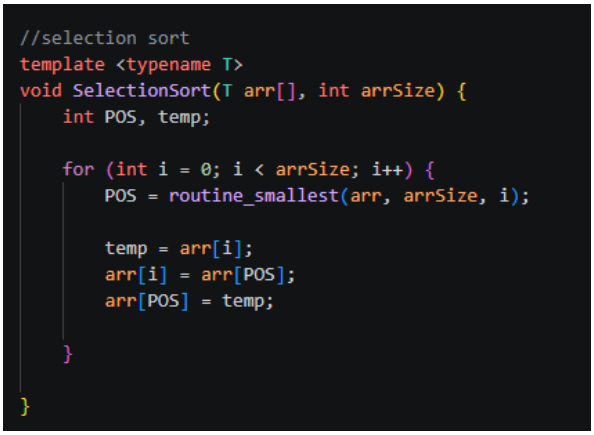
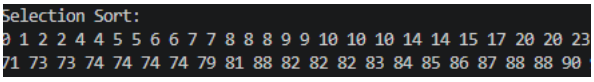
    return 0;
}

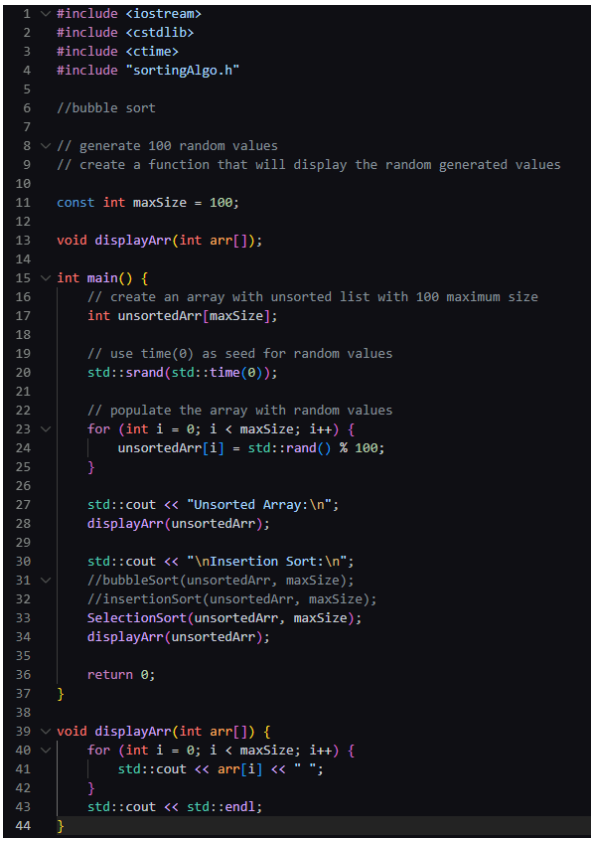
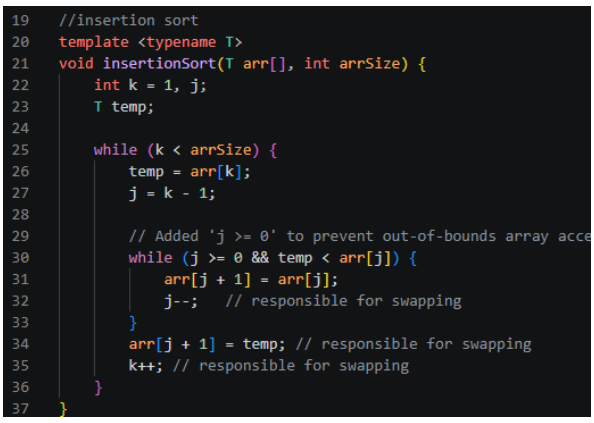
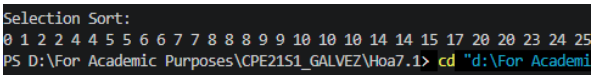
void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```
Unsorted Array:
87 5 40 15 65 95 10 42 88 63 91 92 77 15 51 31 78 4 89 2 91 75 27 93 74 34 69 46 82 18 74 80 34 59 69 23 42 27 30 4 86 35
59 11 21 45 64 80 81 49 47 10 33 17 94 48 10 63 76 43 0 10 26 6 96 58 54 18 12 65 21 65 31
```

| Table 7-2 | Bubble Sort Technique |
|---|---|
| Code and Output: |   |
| Observation: | The code sorts the array with a bubbleSort() function from a custom header file. It runs two loops to compare adjacent numbers and swap them if they are out of order, successfully printing all 100 numbers lined up from lowest to highest. |

| Table 7-3 | Selection Sort Algorithm |
|---|---|

| | |
|---|---|
| Code and Output: |  <br><br>  <br><br>  <br> |
| Observation: | The code sorts the array by using a helper function to find the smallest number in the unsorted section and swap it into its correct place at the front. The final output shows all elements lined up from lowest to highest, proving the find-and-swap logic was successful. |

| Table 7-4. | Insertion Sort Algorithm |
| --- | --- |
| Code and Output: |    |

```
Selection Sort:
0 1 2 2 4 4 5 5 6 6 7 7 8 8 8 9 9 10 10 10 14 14 15 17 20 20 23 24 25
PS D:\For Academic Purposes\CPE21S1_GALVEZ\Hoa7.1> cd "d:\For Academi
```

| | |
|---|---|
| Observation: | The code sorts the list using insertionSort(), which takes numbers one at a time, compares them to previous elements, shifts larger values to the right, and drops each number into its proper spot. The console output shows the numbers neatly organized from lowest to highest, proving the shift-and-insert approach works correctly. |

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**Main.cpp**

```cpp
#include <iostream>
#include "sortingAlgo.h"

const int maxSize = 100;

int val = 12345;

int myRand() {
    val = (1103515245 * val + 12345) % 2147483648;
    if (val < 0) {
        val = -val;
    }
    return val;
}

void displayArr(int arr[]);

int main(){

    //creates an array with unsorted list with 100 maximum size
    int unsortedArr[maxSize];

    //populates the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = (myRand() % 5) + 1;
    }
    displayArr(unsortedArr);

    std::cout << "Sorted Array: " << std::endl;
    selectionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    processAndCountVotes(unsortedArr, maxSize);

    return 0;
}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

**sortingAlgo.h**

```cpp
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <iostream>
#include <string>

template <typename T>
int routine_smallest(T A[], int maxSize, int k) {
    int position = k;
    T smallestElem = A[k];

    for (int j = k + 1; j < maxSize; j++) {
        if (A[j] < smallestElem) {
            smallestElem = A[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize) {
    int POS;
    T temp;

    for (int i = 0; i < arrSize; i++) {
        POS = routine_smallest(arr, arrSize, i);

        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
    }
}

void processAndCountVotes(int arr[], int maxSize) {
    int voteCounts[6] = {0}; // Indices 1 to 5 store votes

    for (int i = 0; i < maxSize; i++) {
        voteCounts[arr[i]]++;
    }

    std::string names[6] = {
        "",
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };
```

```
50
51      int winningCandidate = 1;
52      int highestVotes = voteCounts[1];
53
54      std::cout << std::endl << "Votes Results: " << std::endl;
55      for (int i = 1; i <= 5; i++) {
56          std::cout << "Candidate " << i << " (" << names[i] << "): " << voteCounts[i] << " votes" << std::endl;
57          if (voteCounts[i] > highestVotes) {
58              highestVotes = voteCounts[i];
59              winningCandidate = i;
60          }
61      }
62
63      std::cout << std::endl << "Winner: Candidate " << winningCandidate << " with " << highestVotes << " votes!" << std::endl;
64  }
65
66  #endif
```

| Output Console Showing Sorted Array | Manual Count | Count Result of Algorithm |
|---|---|---|
| Sorted Array:<br>**1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5** | Candidate 1 = 24 votes<br>1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1<br>Candidate 2 = 19 votes<br>2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2<br>Candidate 3 = 22 votes<br>3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3<br>Candidate 4 = 16 votes<br>4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4<br>Candidate 5 = 19 votes<br>5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 | Votes Results:<br>Candidate 1 (Bo Dalton Capistrano): 24 votes<br>Candidate 2 (Cornelius Raymon Agustin): 19 votes<br>Candidate 3 (Deja Jayla Banaga): 22 votes<br>Candidate 4 (Lalla Brielle Yabut): 16 votes<br>Candidate 5 (Franklin Relano Castro): 19 votes<br><br>Winner: Candidate 1 with 24 votes!<br>PS D:\For Academic Purposes\CPE21S1 |

**Summary of lessons learned**
I got to practice using Selection, Insertion, and Bubble Sort to organize numbers from lowest to highest. Selection Sort finds the smallest value and moves it to the front, Insertion Sort slides each number into its spot one by one, and Bubble Sort swaps side-by-side numbers until everything is in order. Instead of using standard built-in functions, I built my own random number generator with a math formula, and I used sorted arrays to make counting election votes a lot easier.

**Analysis of the supplementary activity**
For the supplementary activity, I built an election simulation that processed 100 random votes across five candidates. In main.cpp, I initialized an array, filled it with random vote IDs from 1 to 5, and printed the raw data. I then used selectionSort() to group identical votes together so processAndCountVotes() could count each candidate's total, display the breakdown, and declare the winner.

**Concluding statement / Feedback:**
Overall, I think I did great in this lab. I split the code across multiple files, sorted the data properly, and got the right results on the screen. To keep improving, I just need a bit more practice writing standard C++ templates and figuring out how to handle a tie if two candidates get the same amount of votes.

**3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT**

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Idea Generation**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **I used Gemini to help brainstorm ideas for the code, keeping my core concepts intact. I confirm that my AI usage complies with TIP's AI policy.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.